aikido

10 March 2026

Penetration Test Report

For Parolica

Table of contents

1. Executive summary

1.1 Confidentiality statement 3
1.2 Report Overview 3

2. Findings

2.1 Vulnerability Distribution 4
2.2 Master Findings Table 5
2.3 Detailed Findings 6

Appendices

A. Scope & Methodology 10
B. Vulnerability Coverage 11
C. Glossary 12

1. Executive Summary

1.1 Confidentiality statement

This document is the exclusive property of **Parolica** and **Aikido**. This document contains proprietary and confidential information. Duplication, redistribution, or use, in whole or in part, in any form, requires the consent of Parolica. Parolica may share this document with auditors under non-disclosure agreements to demonstrate penetration test requirement compliance.

1.2 Report Overview

On **10 March 2026**, Aikido conducted a **white box** web application penetration test for Parolica focused on the application(s) within the defined scope and their exposure to targeted attacks. The primary goal of this engagement was to proactively discover vulnerabilities within those scoped applications that could lead to compromise or exposure of sensitive information.

The penetration test simulated real-world attack techniques to identify risks within the scoped application(s). The configured scope was limited to the following domains and associated endpoints:

- **https://parolica.com**

The evaluation resulted in 2 medium and 1 low findings and revealed that there are several opportunities to strengthen the cybersecurity posture of the application. The environment shows a foundational commitment to security, but targeted improvements are recommended to reduce the likelihood and impact of compromise.

Identified Improvement Points

The application currently relies on client-supplied headers to determine user identity for certain security controls, which can allow those controls to be bypassed without proper server-side enforcement. Reviewing how IP-based session and authentication validations are handled would help ensure these mechanisms cannot be influenced by end users.

Authentication endpoints would benefit from request throttling and rate-limiting controls to reduce the risk of automated login attempts. Introducing these protections is a straightforward step that meaningfully raises the effort required for unauthorized access attempts.

Some unauthenticated endpoints may unintentionally reveal whether a given user account exists within the system. Standardizing responses across these flows would help reduce the amount of information available to unauthenticated parties during reconnaissance.

Conclusion

The current security posture reflects a solid baseline, with opportunities to further improve resilience through prioritized remediation and verification. We recommend following the detailed remediation guidance in this report and validating fixes through structured retesting.

2. Findings

2.1 Vulnerability Distribution

Critical	High	Medium	Low
0	0	2	1

The table below outlines each identified vulnerability, categorized by severity and current remediation status. Findings highlight several critical and high-risk issues that could allow attackers to compromise application integrity, access sensitive data, or execute arbitrary code on the server.

- **Medium-severity issues** include client-controlled X-Forwarded-For headers undermining IP-bound session/OTP validation and the absence of rate limiting on the login endpoint, which could allow attackers to bypass security controls and conduct credential stuffing attacks.
- **Low-severity issues** involve user enumeration via an unauthenticated endpoint that exposes whether accounts exist, potentially aiding targeted attacks.

2.2 Master Findings Table

ID	Title	State	Severity
PT-1	Client-controlled X-Forwarded-For breaks IP-bound session/OTP validation	Fixed	Medium
PT-2	No Rate Limiting on /api/authentication/log-in Enables Credential Stuffing	Fixed	Medium
PT-3	User Enumeration via /api/authentication/fetch-salt (Unauthenticated)	Fixed	Low

2.3 Detailed Findings

2.3.1 PT-1 - Client-controlled X-Forwarded-For breaks IP-bound session/OTP validation

~~Medium~~ Resolved

Identified on: March 10th 2026 | Resolved on: March 17th 2026

Retest summary

The original reproduction that relied on client-controlled X-Forwarded-For no longer works. We captured a valid X-Session-Token from the authenticated browser session and observed that sending an Administration request with a spoofed X-Forwarded-For did not cause the expected 401; instead the endpoint returned 200. Source review confirms the gateway now forwards x-ip-address from X-Real-IP rather than X-Forwarded-For, addressing the original root cause.

Original finding

The API gateway forwards the client-supplied X-Forwarded-For header as x-ip-address to backend services, and the server uses it to enforce IP binding for sessions and OTP tokens. Because X-Forwarded-For is attacker-controlled, an attacker with a stolen token can spoof the victim's IP and bypass the intended protection.

Business impact

This vulnerability weakens authentication protections by allowing bypass of IP-bound token checks, making session hijacking and OTP flow abuse more feasible once a token is obtained. Attackers can replay stolen sessions or complete 2FA with forged IPs, resulting in account takeover, unauthorized key management actions, and data exposure. It also degrades the integrity of audit logs and security analytics by allowing spoofed IPs. While exploitation requires possession of a valid token and knowledge of the victim IP, both are realistic in many threat models (phishing, XSS, shared networks, leaked logs).

How to exploit

Step 1: Log into your own account via the UI and capture a valid X-Session-Token from any Administration API request (e.g., /api/administration/read-keys). Also note your session IP from Security → Recent sessions.

Step 2: Call an authenticated Administration endpoint with a wrong spoofed IP in X-Forwarded-For.

```
curl -i 'https://parolica.com/api/administration/get-recent-sessions' \
  -H 'Accept: application/json' \
  -H 'X-Client-Version: ****47c1' \
  -H 'X-Session-Token: <YOUR_SESSION_TOKEN>' \
  -H 'X-Forwarded-For: 8.8.8.8' \
  -H 'User-Agent: PoC/1.0'
```

Step 3: Repeat the exact same request but set X-Forwarded-For to your stored IP.

```
curl -i 'https://parolica.com/api/administration/get-recent-sessions' \
  -H 'Accept: application/json' \
  -H 'X-Client-Version: ****47c1' \
  -H 'X-Session-Token: <YOUR_SESSION_TOKEN>' \
  -H 'X-Forwarded-For: <YOUR_STORED_IP>' \
  -H 'User-Agent: PoC/1.0'
```

Step 4: Optional 2FA variant: with 2FA enabled, use the authn_key returned from /api/authentication/log-in and call /api/authentication/provide-otp twice: first with wrong X-Forwarded-For (expect UNAUTHENTICATED), then with the stored IP (expect success).

```
curl -i 'https://parolica.com/api/authentication/provide-otp' \
  -H 'Accept: application/json' \
  -H 'X-Client-Version: ****47c1' \
  -H 'X-Forwarded-For: <WRONG_OR_CORRECT_IP>' \
  -H 'Content-Type: application/json' \
  --data '{"authnKey":"<AUTHN_KEY>","otp":"000000","yieldTrustedToken":false}'
```

Remediation

Implement the following measures to eliminate this vulnerability:

- Do not read X-Forwarded-For from public requests in grpc_gateway/main.go. Instead, derive the client IP from the TCP connection (RemoteAddr) or from a trusted header inserted by your edge proxy after stripping any inbound X-Forwarded-For.
- At the edge (e.g., Cloudflare/Nginx), always overwrite and sanitize X-Forwarded-For; optionally forward a canonical header (e.g., X-Real-IP) only to internal services.
- Avoid using client IP as a hard requirement for token validation. Treat IP as advisory, or combine with non-spoofable bindings (device-bound tokens, signed cookies, WebAuthn).
- Add monitoring to detect suspicious IP header usage and ensure audit logs are not populated from spoofable headers.

2.3.3 PT-2 - No Rate Limiting on /api/authentication/log-in Enables Credential Stuffing

~~Medium~~ Resolved

Identified on: March 10th 2026 | Resolved on: March 23rd 2026

Retest summary

The /api/authentication/log-in endpoint now enforces server-side rate limiting per username. After ~8 rapid attempts within 10 minutes, subsequent requests return an application-level RATE_LIMITED error (HTTP 200). I could not reproduce the original behavior of unlimited 200 INVALID_CREDENTIALS responses.

Original finding

The /api/authentication/log-in endpoint accepts unlimited authentication attempts without any throttling, lockout, or backoff. This allows attackers to perform credential stuffing and brute-force attacks at scale because the server returns 200 responses for rapid repeated attempts with no 429 or delay.

Business impact

Unbounded login attempts materially increase the feasibility of credential stuffing and brute-force attacks, leading to potential account takeovers, user lockouts via password resets, and support/SOC overhead. If compromised accounts contain personal data, this can raise compliance concerns (e.g., GDPR/CCPA). Lack of throttling also facilitates automated abuse probing and reduces the signal available for detection.

How to exploit

Step 1: Send multiple rapid login attempts to the authentication endpoint using the same invalid credentials

```
import requests, json, time
url = 'https://parolica.com/api/authentication/log-in'
headers={'Content-Type':'application/json'}
payload = {'username':'nonexistent_bruteforce','digest':'A'*43}
statuses=[]
for i in range(20):
    r = requests.post(url, headers=headers, data=json.dumps(payload), timeout=20)
    statuses.append((r.status_code, r.text.strip()))
    time.sleep(0.05)
print('Unique statuses:', sorted({s for s,_ in statuses}))
print('\n'.join(f'{i}: {s} {b}' for i,(s,b) in enumerate(statuses[:5])))
```

Step 2: Observe there is no 429 Too Many Requests or any server-enforced delay/lockout despite 20 rapid attempts

Remediation

Implement the following measures to eliminate this vulnerability:

- Introduce per-IP and per-username sliding-window rate limits for /api/authentication/log-in, returning 429 Too Many Requests when thresholds are reached
- Implement temporary account-level lockouts after N consecutive failures, with safe reset policies (time-based unlock, CAPTCHA, or out-of-band verification)
- Add exponential backoff or fixed minimal delays after each failed attempt to slow automated attacks
- Emit security telemetry for failed attempts and bans; monitor and alert on brute-force patterns
- Optionally require CAPTCHA after repeated failures from the same IP/ASN or against the same username

2.3.4 PT-3 - User Enumeration via /api/authentication/fetch-salt (Unauthenticated)

~~Low~~ Resolved

Identified on: March 10th 2026 | Resolved on: March 23rd 2026

Retest summary

The user enumeration via /api/authentication/fetch-salt appears fixed. Both existing and non-existing usernames now return HTTP 200 with {"error":"NONE","salt":"<argon2id params>"} and identical headers/content lengths, preventing differentiation of account existence.

Original finding

The unauthenticated /api/authentication/fetch-salt endpoint returns distinct responses for existing vs non-existing usernames. This enables reliable user enumeration by revealing whether an account exists before attempting authentication.

Business impact

Exposing account existence significantly improves the efficiency of credential stuffing and targeted brute-force attacks, increasing the likelihood of account takeover. Verified username lists can be cross-referenced with breached password corpora, elevating organizational risk, potential data exposure, and incident response load. While no credentials are directly leaked, this weakness meaningfully lowers the bar for successful compromise.

How to exploit

Step 1: Send a POST request for a known existing username to fetch its salt (no auth required)

```
curl -i -s -X POST https://parolica.com/api/authentication/fetch-salt \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json' \
  --data '{"username":"keyring_aikido"}'
```

Step 2: Send a POST request for a random, non-existent username

```
curl -i -s -X POST https://parolica.com/api/authentication/fetch-salt \
  -H 'Content-Type: application/json' \
  -H 'Accept: application/json' \
  --data '{"username":"nonexist_1234abcd"}'
```

Step 3: Observe the different responses that clearly reveal whether the username exists

Remediation

Implement the following measures to eliminate this vulnerability:

- Always return a generic success response for both existing and non-existing users; for non-existing users provide a constant/dummy Argon2 parametrization so that subsequent login still fails but enumeration is prevented
- Alternatively combine salt retrieval and login into a single call that always yields a generic INVALID_CREDENTIALS on failure
- Introduce strict rate limiting and IP throttling on the fetch-salt endpoint to hinder automated enumeration
- Add monitoring/alerting for high-rate access patterns to this endpoint

Appendices

A. Scope & Methodology

Methodologies

The penetration test utilized automated security analysis combined with industry-standard penetration testing methodologies

- AI-Powered Security Analysis Engine
- OWASP Testing Guide v4.2
- PTES (Penetration Testing Execution Standard)
- NIST SP 800-115
- Automated Vulnerability Assessment (OWASP ASVS)

Penetration test types

	Black box	Grey box	White box
Goal	Assess security defences against a cyber attack	Assess security defences against a cyber attack	Assess security defences against a cyber attack
Access level	Zero access or internal information	Some internal access and internal information	Complete open access to applications and systems
Pros	Most realistic Testing is performed from point of view of attacker	More efficient and complete than Black Box while saving time and money Testing is performed from point of view of attacker	More comprehensive, less likely to miss a vulnerability Testing is performed from point of view of attacker
Cons	Time consuming and more likely to miss a vulnerability	More hidden vulnerabilities might be missed	More action required by client to provide information and more time necessary to process documents

B. Vulnerability Coverage

OWASP Top 10 Compliance

The penetration test fully covers all vulnerability categories defined in the OWASP Top 10, ensuring detection of the most critical web application security risks.

Tested Vulnerability Classes

The penetration test explicitly checked for the following vulnerability types:

Injection & Execution	Authentication & Access Control
SQL, NoSQL, XPath, & LDAP Injection	Broken or Improper Authentication
Cross-Site Scripting (XSS)	Missing Authentication for Critical Functions
Server-Side Request Forgery (SSRF)	Insecure Direct Object Reference (IDOR)
Server-Side Template Injection (SSTI)	Improper Access Control
Local File Inclusion (LFI)	Cross-Site Request Forgery (CSRF)
Insecure Deserialization	Excessive Authentication Attempts

Data Security & Logic	Configuration, Files & Cryptography
Business Logic Flaws	Unrestricted File Uploads
Exposure of Sensitive Information	External Control of File Name or Path
Information Exposure via Errors	Open Redirects
Directory Listing Enabled	Improper JWT Signature Verification
Web Cache Poisoning	Broken or Risky Cryptographic Algorithms
Insufficient Data Authenticity	Hard-Coded Passwords or Credentials
Reliance on Cookies Without Integrity	Insufficiently Protected Credentials

C. Glossary

Authentication	The process of verifying the identity of a user, device, or system before granting access to resources.
Authorization	The process of determining what permissions an authenticated user has and what resources they can access.
CVSS	Common Vulnerability Scoring System - A standardized method for rating the severity of security vulnerabilities.
IDOR	Insecure Direct Object Reference - When an application provides direct access to objects based on user-supplied input without proper authorization checks.
PII	Personally Identifiable Information - Any data that could potentially identify a specific individual.
SQL Injection	A code injection technique where malicious SQL statements are inserted into application data entry points.
XSS	Cross-Site Scripting - A vulnerability that allows attackers to inject malicious scripts into web pages viewed by other users.
WAF	Web Application Firewall - A security device that monitors, filters, and blocks HTTP traffic to and from a web application.